参考 **Portfolio Optimization — Qiskit Finance documentation**。

n 个投资品的均值方差投资组合优化：

$$\min_{x\in\{0,1\}^n} qx^T\Sigma x - \mu^T x$$
$$\text{subject to: } 1^T x = B$$

- $x \in \{0,1\}^n$ 表示投资方案，$x_i = 1$ 表示选择第 i 个投资品，$x_i = 0$ 反之；
- $\mu \in \mathbb{R}^n$ 表示资产预期收益；
- $\Sigma \in \mathbb{R}^{n\times n}$ 表示指定资产间的协方差矩阵；
- $q > 0$ 是投资人的风险偏好；
- B 表示预算。

在上面的问题中，有两个假设：

- 所有资产的价格都为 1；
- 预算 B 全部投入。

将问题结果映射为哈密顿量，其基态代表着最优解，可用QAOA解决。

导入包，主要用到 `qiskit` 以及 `qiskit[finance]`。

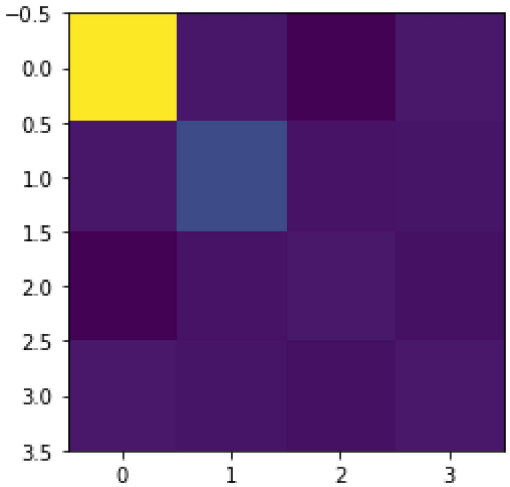
In [ ]:
```
from qiskit import Aer
from qiskit.algorithms import QAOA, NumPyMinimumEigensolver
from qiskit.algorithms.optimizers import COBYLA
from qiskit.utils import QuantumInstance
from qiskit_finance.applications.optimization import PortfolioOptimization
from qiskit_finance.data_providers import RandomDataProvider
from qiskit_optimization.algorithms import MinimumEigenOptimizer
from qiskit.utils import algorithm_globals
from qiskit_optimization.converters import QuadraticProgramToQubo
import numpy as np
import matplotlib.pyplot as plt
import datetime
```

生成随机数据。

In [ ]:
```
# 资产数
num_assets = 4
# 随机种子
seed = 121
# 生成数据
stocks = [f'TICKER{i}' for i in range(num_assets)]
data = RandomDataProvider(tickers=stocks, start=datetime.datetime(2016, 1, 1), end=datetime.datetime(2016, 1, 30), seed=seed)
data.run()
# 均值
mu = data.get_period_return_mean_vector()
# 协方差矩阵
sigma = data.get_period_return_covariance_matrix()
```

In [ ]:
```
plt.imshow(sigma)
```

Out[ ]:
```
<matplotlib.image.AxesImage at 0x1daf48071c0>
```



定义投资组合问题 `PortfolioOptimization`。

In [ ]:
```
# 风险因子
q = 0.5
# 预算（至多持有投资品数量）
budget = num_assets // 2
portfolio = PortfolioOptimization(expected_returns=mu, covariances=sigma, risk_factor=q, budget=budget)
qp = portfolio.to_quadratic_program()
qp
```

Out[ ]:
```
<QuadraticProgram: minimize 0.00346068382831024 1*x_0^2 + 0.0001430025383249795..., 4 variables, 1 constraints, 'Portfolio optimization'>
```

定义两个函数，用于格式化输出结果。

In [ ]:
```
def index_to_selection(i, num_assets):
    s = f'{i:b}'.rjust(num_assets)
    x = np.array([1 if s[i] == '1' else 0 for i in reversed(range(num_assets))])
    return x


def print_result(result):
    selection = result.x
    value = result.fval
    print(f'Optimal: selection {selection},value {value:.5f}')
    eigenstate = result.min_eigen_solver_result.eigenstate
    eigenvector = eigenstate if isinstance(eigenstate, np.ndarray) else eigenstate.to_matrix()
    probabilities = np.abs(eigenvector) ** 2
    i_sorted = reversed(np.argsort(probabilities))
    print("\n----------------- Full result ---------------------")
    print("selection\tvalue\t\tprobability")
    print("---------------------------------------------------")
    for i in i_sorted:
        x = index_to_selection(i, num_assets)
        value = QuadraticProgramToQubo().convert(qp).objective.evaluate(x)
        probability = probabilities[i]
        print("%10s\t%.4f\t\t%.4f" % (x, value, probability))
```

首先用经典方法得到结果。

In [ ]:
```
exact_mes = NumPyMinimumEigensolver()
exact_eigensolver = MinimumEigenOptimizer(exact_mes)
result = exact_eigensolver.solve(qp)
print_result(result)
```

```
Optimal: selection [0. 0. 1. 1.],value -0.00576

----------------- Full result ----------------------
selection       value          probability
----------------------------------------------------
 [0 0 1 1]      -0.0058        1.0000
 [1 1 1 1]      4.0777         0.0000
 [0 1 1 1]      1.0161         0.0000
 [1 0 1 1]      1.0197         0.0000
 [1 1 0 1]      1.0283         0.0000
 [0 1 0 1]      0.0024         0.0000
 [1 0 0 1]      0.0064         0.0000
 [0 0 0 1]      1.0167         0.0000
 [1 1 1 0]      1.0246         0.0000
 [0 1 1 0]      -0.0008        0.0000
 [1 0 1 0]      0.0028         0.0000
 [0 0 1 0]      1.0136         0.0000
 [1 1 0 0]      0.0113         0.0000
 [0 1 0 0]      1.0217         0.0000
 [1 0 0 0]      1.0256         0.0000
 [0 0 0 0]      4.0722         0.0000
```

用QAOA得到结果。

In [ ]:

```python
algorithm_globals.random_seed = 1234
backend = Aer.get_backend('statevector_simulator')
cobyla = COBYLA()
cobyla.set_options(maxiter=250)
quantum_instance = QuantumInstance(backend=backend, seed_simulator=seed, seed_transpiler=seed)
qaoa_mes = QAOA(optimizer=cobyla, reps=3, quantum_instance=quantum_instance)
qaoa = MinimumEigenOptimizer(qaoa_mes)
result = qaoa.solve(qp)
print_result(result)
```

```
Optimal: selection [0. 0. 1. 1.],value -0.00576

----------------- Full result ----------------------
selection       value           probability
----------------------------------------------------
 [0 0 1 1]      -0.0058         0.1687
 [0 1 1 0]      -0.0008         0.1675
 [0 1 0 1]      0.0024          0.1667
 [1 0 1 0]      0.0028          0.1666
 [1 0 0 1]      0.0064          0.1658
 [1 1 0 0]      0.0113          0.1647
 [0 0 1 0]      1.0136          0.0000
 [0 0 0 0]      4.0722          0.0000
 [1 1 1 1]      4.0777          0.0000
 [1 1 0 1]      1.0283          0.0000
 [1 1 1 0]      1.0246          0.0000
 [0 0 0 1]      1.0167          0.0000
 [1 0 1 1]      1.0197          0.0000
 [0 1 0 0]      1.0217          0.0000
 [1 0 0 0]      1.0256          0.0000
 [0 1 1 1]      1.0161          0.0000
```

测量得到投资方案 $x=[0,0,1,1]$ 概率最高，这与经典方法求得的结果相符合。